

1- Introduction

- Computer science is the science of automatic information processing.
- The computer is a machine capable of automatically performing arithmetic and logical operations from programs defining the sequence of these operations.

- **Why the computer?**

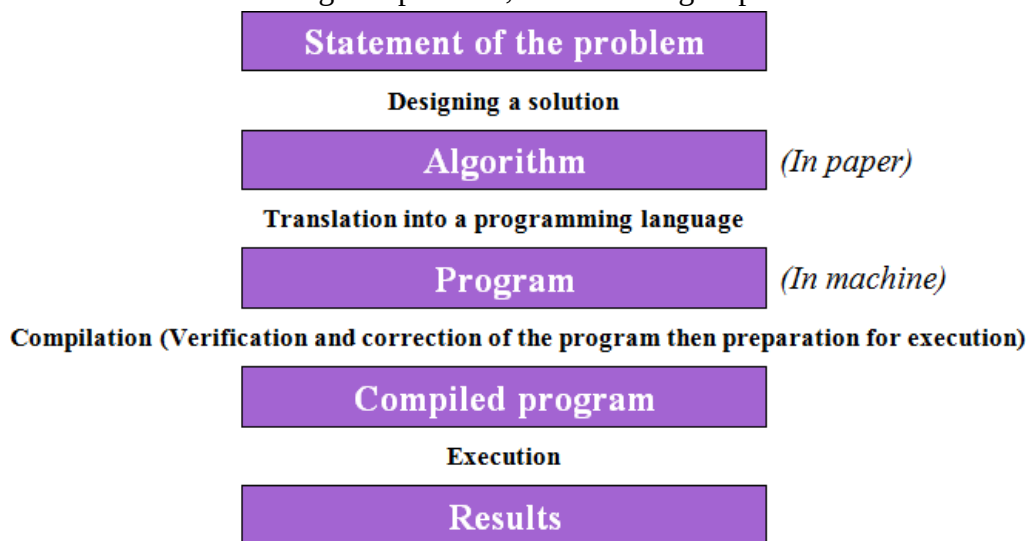
The following table represents a small comparison between the human being and the computer.

a human	Computer
Intelligent	Not intelligent
Slow, can be wrong, small memory	Fast, exact calculation, very large memory

We see that the advantage of one constitutes the disadvantage of the other and vice versa. From there the objectives of the algorithmic course is to bring the intelligence of the human being to be able to solve problems towards the machine to have correct results, in a fast way, whatever the size of the problem

2. Steps for solving a problem

To solve a given problem, the following steps must be followed:

**Example**

```

Algorithm Average ;
Var      x, y, Avg : real ;
Begin
Write("Give me two values");
Read(x, y) ;
Avg ← (x+y)/2 ;
Write ("The average is : ", Avg);
End
  
```

```

//Program Average ;
#include <stdio.h>
int main(){
float x, y, Avg;
printf('Enter two values : ');
scanf('%f%f', &x, &y) ;
Avg=(x+y)/2 ;
printf('The average is %f : ', Avg) ;
return 0;
}
  
```

3. Definitions

3.1 Algorithm : is a sequence of ordered actions (operations or steps) which makes it possible to solve a given problem, such as for example the steps of solving a quadratic equation.

3.2 Language algorithmique : is a set of rules that allows us to write an algorithm correctly.

3.3 Programme : is a sequence of instructions written in a programming language, it is the translation of the algorithm into a programming language.

Examples of programming languages: C, C++, Java, Python....

Exercise: Given a circle of radius r . Write an algorithm that reads the value of r then calculates and displays the area of the corresponding circle.

Solution :

(1) Problem specification

Data : The radius r (real) → **Results :** The area A (real)

General diagram of the solution :



(2) Problem analysis $A = 3.14 * r * r$

(3) Writing the corresponding algorithm

Algorithm Circle_Area ;

Const pi=3.1416;

Var

r, s : real ; /* r : radius, a : area*/

Begin

/*Data reading*/

Print("Give me the radius");

Read(r) ;

/*Result calculation*/

$a \leftarrow pi * r * r$; /* or : $a \leftarrow pi * r \uparrow 2$; */

/*Result display*/

Print ("The area is : ", a);

End

4. The algorithmic language

- It is a language very close to natural language and independent of the chosen programming language.
- An algorithm is as follows:

Algorithm name of algorithm ;
 Declarative part;
Begin
 Actions part ;
End.

- The objects: (constants, variables) that an algorithm manipulates must be defined (or declared) before their use.
- Any instruction (action) is followed by a “;” to separate it from another action.
- Comments can be located anywhere in the algorithm, they are used by the programmer to clarify the algorithm and not by the machine.
- A comment is enclosed in /* */.

4.1 Concept of declaration:

Syntaxe :

Var identifier: type ;
Const identifier = value ;

Const : is an object with a value that cannot be modified.

Example : Constant Pi = 3.141559

Var (Variable) : is an object whose value can be modified at any time. She may be :

- an input data;
- a final result of a calculation;
- an intermediate calculation result. (Auxiliary variable) Example: Var x, y: integer; z: real;

• **Identifier:** is the name of a variable, constant, algorithm or sub-algorithm. An identifier begins with an alphabetical letter, followed or not by alphabetic letters, numbers, or “_”.

Example: Ali, name, x1, avg, Avg_1 are valid identifiers.

#B, A.X, 1B2, x 1 are **incorrect** identifiers.

• Types of Values :

Const identifier = value ;

We have 5 basic types :

Integer : set of integer numbers like : 2, -5, 0,

Real : set of real numbers like : 2.4, -5, 0, -0.3

Boolean : two values only are possible : true, false

Character : any number, or letter or symbol in one position putted in “ ” is considered as character value.

Example : ‘a’, ‘4’, ‘)’, ‘#’, ‘+’, ‘ ‘,

String : is a set of characters putted between “ ”.

Example : “good morning”, “g4+[@,”,

- **Basic types :**
Var identifier : Type ;

1) Integer type : This data type is used to store whole numbers. i.e. values with no decimal places. It can be positive, negative or null.

■ **Example of an integer variable :** age, number_of_student...

2) Real type : This data type is used to store real numbers i.e. values that might have decimal places. A real number has this form: integer part.decimal part preceded or not by a sign.

■ **Example of a real variable :** weight, height, distance....

3) Boolean type : This data type is used to store the logical data TRUE/FALSE

■ **Example of a boolean variable:** married, present, exist.

4) Character type : This data type characterizes the domain of alphabetic characters (A, B, ..., Z, a, b, ..., z), numbers (0,...,9), symbols (, ; () [] . ..), arithmetic operators (+, -, /, *) and comparison operators (<, >, =). A character is a single letter, number or symbol from above and is treated as a text, even if it is a number.

■ **Example of a character variable :** study_section, driving_license category, ...

5) String type : A string is made of 0, one, two or more characters

■ **Example of a string variable :** Decision, Name, Address, Season....

5.2) Expression :

5.2.A) Arithmetic expression :

- Is a finite sequence formed of arithmetic operators on variables or numeric values and which has a numeric value.
- **Arithmetic operators :**

Operator	Type operands and meaning	Examples
↑	Real, integer (rise to power)	$x^2 \rightarrow x \uparrow 2$, $\sqrt{x} = x^{1/2} = x \uparrow (1/2)$
/	Real, Real (real division)	$5/2=2.5$
*	Real, integer (multiplication)	$5*3$

+	Real, integer (addition)	5+2		
-	Real, integer (subtraction)	4-1		
DIV	integer (quotient of integer division)	9 div 3 = 3	13 div 5 = 2	3 div 5=0
MOD	integer (modulo: remainder of integer division)	9 mod 3 =0	13 mod 5 =3	3 mod 5=3

 *Examples of arithmetic expressions :* $(x-y)*z$, $x \uparrow 2 - 3*t$

5.2.B) Expression booléenne :

- Can contain logical operators (or, and, not), arithmetic operators (+, -, /, *, mod, div, $\uparrow$), as well as comparison operators (<, >, =, <>, >=, <=).
-
- **Logical operators :**

A	B	A and B	A or B	not A
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

 *Examples of boolean expressions :*

$(x+y)<10$, $\text{not}((x<5) \text{ or } (y>=3)) \Leftrightarrow (x>=5) \text{ and } (y<3)$

6. Concept of action:

An action is an event that acts on the objects of the algorithm by modifying their values or by observing them.

6.1. Assignment:

It consists of placing a value in a variable belonging to its domain. It is represented by the sign ' $\leftarrow$ '.

Syntax: three forms are possible:

Identifier $\leftarrow$ value;

Identifier $\leftarrow$ Identifier;

Identifier $\leftarrow$ expression;

Examples: $x \leftarrow 3$;

$y \leftarrow x$; {reads “y receives x”}

$y \leftarrow (x+y)/z$; $A \leftarrow (x>2)$;

The expression can be arithmetic or Boolean. The value of the expression must conform to the domain of the assigned variable.

 **Problem 1 :** Unroll this portion of the algorithm.

```

y      ←      2 ;
x      ←      y ;
x      ←      x*2 ;
x1     ←      y + x ;
y1     ←      x1+2 ;
y1     ←      y1*2 ;
x      ←      y1+3 ;

```

Solution :

Instruction	y	x	x1	y1
$y \leftarrow 2$;	2	?	?	?
$x \leftarrow y$;	2	2	?	?
$x \leftarrow x*2$;	2	4	?	?
$x1 \leftarrow y+x$;	2	4	6	?
$y1 \leftarrow x1+2$;	2	4	6	8
$y1 \leftarrow y1*2$;	2	4	6	16
$x \leftarrow y1+3$;	2	19	6	16

Remark : you can assign an integer type value to a real type variable, but not vice versa..

6.2. Writing (Display) :

Syntaxe :

Print (list of objects) ; or write (list of objects) ;

- This instruction simply performs the display of the values of the objects mentioned in the list. Objects are separated by ','

- An object can be :
A value : like 5, "hello",.... **A value is displayed as it is.**
A variable : like x, y.... **It is the value of the variable that is displayed.**
An expression : like (x+y)/2, A or B,... **The expression is evaluated and the result is displayed.**

 Example :

```

a ← 5 ;
Print (a) ;                /*5*/
c ← 4 ;
Print ("c=",c) ; /*c=4*/
b ← a ↑ 2 - c ;
Print (a ↑ 2 - c) ;        /*21*/
Print (a, b, c) ;          /*5214*/
Print (a, b, c) ;          /*5214*/
Print (a, " ", b, " ", c) ; /*5
21 4*/
Print ("a=", a, " b=", b, " c=", c) ; /*a=5 b=21 c=4*/

```

Remark : It is important to precede any display of a value of a certain variable with a message so that the user understands the values displayed (as in the last display).

6.3. Reading :

This instruction reads a value from an input device (usually the keyboard) and then stores it in a variable in memory. If you have several variables to read, just separate them with ',' for example: read(x , y);

Syntaxe :

Read (list of variables) ;

 Example :

Read(x); ⇔ x ← The value entered by the user;

```

x ← - 4;      /*x has the value - 4*/
Print ("Give me a value : " ) ;
Read(x);      /*if the user enters the value 5*/
Print("x=", x); /*this displays : x=5*/
/*because x received the value 5 which overwriten the old value -4*/

```

 **Remark 1 :** it is important to precede a read instruction by a message display instruction to indicate to the user the nature and type of the value to be entered.

 **Remark 2 :**

Case 1	Case 2
<pre>.... x ← 3 ; y ← 5 ; Sum ← x + y ; Print (Sum) ;</pre>	<pre>..... Print ("Give the value of x : ") ; Read (x) ; Print ("Give the value of y : ") ; Read (y) ; Sum ← x + y ; Print (Sum) ;</pre>

(Oral)

Here the two portions of the algorithm make it possible to calculate and display the sum of two integers (x, y) but:

* In case 1, the values of x and y are fixed at 3 and 5, so if we want to calculate the sum of two other values, we are obliged to modify the values assigned to x and y in the algorithm, then execute the algorithm.

* Unlike in case 2, the values of x and y are given by the user, so it is enough to re-execute the algorithm without modifying it and put any values for x and y. We say that the data is parameterized.

The parameterization of variables in the reading, allows an algorithm to be executed several times for different data.

Important:

To write an algorithm we must follow the following steps:

- Specify input data and results (variables)
- Determine the steps for transforming data into results:

***Reading data *Calculating results *Displaying results**